Atividade 3 — persistência, consultas e concorrência

Disciplina: Desenvolvimento Web com .NET — UNIP. **Fonte:** Roteiros.pdf, páginas 9–11.
Consolidação: 29/10/2026.

Objetivo e ferramentas

Persistir tarefas, recriar esquema por migrations, executar seed idempotente, manter auditoria atômica e detectar concorrência. São utilizados EF Core 10, provider SQL Server, ADO.NET e testes xUnit em banco exclusivo. `UserSecretsId` permite manter a conexão de desenvolvimento fora do repositório; o README apresenta o comando de configuração.

O domínio cumulativo utiliza `Tarefa`, `Usuario : IdentityUser` e `Auditoria`. `PortalTarefasDbContext` deriva de `IdentityDbContext<Usuario>` e recebe somente as opções no construtor, permitindo pooling sem guardar dados do usuário no contexto. O SQL Server mantém os usuários, tarefas e eventos de auditoria em tabelas próprias.

Decisões implementadas

- O banco limita os tamanhos de título, descrição, situação, equipe e identificadores, verifica a prioridade entre 1 e 3 e restringe a situação a Pendente, EmAndamento ou Concluida. Há índices para equipe/situação/prazo, publicação/prazo, proprietário e histórico.
- `rowversion` fornece concorrência otimista. O cliente recebe a versão em Base64 de oito bytes e precisa enviá-la ao alterar ou excluir. Uma versão obsoleta resulta em `ConflitoException`; o serviço não sobrescreve a edição mais recente.
- A criação e sua auditoria utilizam uma transação explícita porque o identificador da tarefa é gerado pelo banco. Alteração e exclusão salvam o recurso e o evento em um único `SaveChangesAsync`, que mantém a atomicidade relacional. `Auditoria.TarefaId` é um identificador histórico sem FK, preservando o registro após exclusão.
- As transações explícitas são executadas pela estratégia do provider, mantendo compatibilidade com repetição de falhas transitórias. Ao repetir uma criação após resultado incerto de commit, o serviço procura o identificador, o instante original e o respectivo evento antes de inserir novamente. A carga de exemplos testa a existência de tarefas dentro de transação serializável.
- A consulta usa `AsNoTracking`, filtros antes da paginação, tamanho máximo 50, `CountAsync`, `Skip`/`Take` e projeção para DTO. Os nomes de ordenação pertencem a uma lista permitida, sem interpolação de SQL. Um identificador desempata a ordenação.
- A visibilidade considera publicação, autenticação e equipe; o administrador autenticado visualiza todas as tarefas. Criação exige login e equipe própria, com exceção administrativa. Edição permite proprietário, integrante da equipe ou claim específica para outra equipe. Exclusão exige o perfil Administrador.

- O painel EF e o painel ADO.NET aplicam os mesmos critérios de visibilidade e de vencimento. A consulta ADO.NET parametriza situação, data, publicação, autenticação, perfil e equipe; a conexão é fechada pelo método somente quando ele próprio a abriu. A data civil usa o fuso de São Paulo.
- A validação cruzada exige prazo e descrição com pelo menos dez caracteres úteis para prioridade alta, tanto no DTO quanto nas chamadas diretas ao serviço.

Modo Memory e carga inicial

`InMemoryTarefaService` é uma alternativa explicitamente selecionada, registrada como singleton. Um lock cobre leitura, alteração, comparação de versões e auditoria, garantindo que somente uma atualização da mesma versão seja aceita. Seus dados são voláteis e o painel ADO.NET informa indisponibilidade, pois esse modo não possui conexão relacional.

`SeedData` cria os perfis Usuario/Administrador de forma idempotente. Contas iniciais dependem de `Seed:AdminEmail`, `Seed:AdminPassword`, `Seed:UserEmail` e `Seed:UserPassword` em configuração externa; não há senha fixa ou registro de credenciais em log. A conta administrativa recebe a claim `tarefas:editar-outra-equipe=true`. Os exemplos são opcionais, ativados por `Seed:SampleData`, e só são incluídos quando não há tarefas.

`DesignTimeDbContextFactory` permite gerar migrações sem iniciar a aplicação. A conexão real pode ser fornecida por `ConnectionStrings__PortalTarefas`; a fábrica não aplica migrações automaticamente.

Verificação e limites

Foram preparados testes em `DomainTests.cs` e `InMemoryTests.cs` para validação cruzada, formato da versão, publicação/equipes, paginação, filtros, lista permitida de ordenação, autorização, colisão entre duas edições, exclusão com versão e preservação da auditoria. A correção executou a suíte completa com SQL Server no banco exclusivo `PortalTarefas_Tests_20261028_dfc3ccbe`, criado para a auditoria e reutilizado nos testes: 55 testes aprovados, zero falhas e zero ignorados. O resultado real está em `docs/evidencias/integracao/20261029T172845759Z-55df9f49/integration.trx` e no `summary.json` do mesmo diretório. Esse número corresponde à suíte cumulativa, não somente a este roteiro.

A aprovação dos testes do modo Memory não comprova conectividade SQL Server, aplicação de migrações, tipo `rowversion` no servidor, rollback relacional ou equivalência real entre os painéis EF/ADO.NET. Essas verificações exigem uma instância SQL Server configurada e disponível. Não se deve registrar esses itens como verificados com base apenas nos testes unitários.

Comparação EF Core e ADO.NET

Aspecto	EF Core no painel	ADO.NET no painel
Código	Projeção LINQ para `PainelDto` e filtros compostos	SQL explícito, parâmetros, conexão e leitura do resultado
Rastreamento	`AsNoTracking`, adequado à consulta	Não há change tracker
Clareza	Próximo ao modelo C# e componível	SQL exato visível, com maior responsabilidade manual

Controle de SQL	Traduzido pelo provider	Comando e tipos dos parâmetros explicitados
Cenário escolhido	CRUD e consultas usuais da aplicação	Comparação didática e leituras que exijam controle específico

A suíte SQL compara os mesmos escopos de visibilidade nas duas consultas. Não foi inferida superioridade de desempenho a partir da quantidade de código ou de uma medição local.

Aprimoramento e conclusão

Em um banco separado de seed (`PortalTarefas_Tests_20261028_dfc3ccbe_seed`), duas inicializações reais conservaram 3 tarefas, 3 auditorias, 2 roles e as mesmas 3 migrations. Os comandos/estados coletados estão em `sql-seed-primeira-execucao.txt`, `sql-seed-segunda-execucao.txt` e `sql-migrations-estado.txt`, em `docs/evidencias`. Essa prova complementa os cenários de seed e concorrência da suíte SQL.

Uma migration adicional cria o índice do painel público. Integrações usam banco reservado, conexão externa e guardas que recusam bancos fora do prefixo de teste. Migrations, versão real, conflitos e equivalência EF/ADO possuem execução própria; os comandos, resultados e limites estão identificados na matriz.